온리티칭 ERD 설계 — 4가지 핵심 질문 답변

작성일 2026-04-08 · 사용자 규모 확장성 / 새 탭 추가 / 매핑 효율 분석

Q1. 이미지처럼 하면 회원 수가 많아져도 병목현상이 안 생기는지?

> 답 — 인덱스 + 비정규화만 잘 되면 사용자 1만명까지 OK. 그 이후는 추가 작업 필요.

안 생기는 이유 (이미 잘된 것)

- ✅ **도메인별 테이블 분리** — 한 테이블이 비대해지지 않음
- ✅ **bigint PK** — 사용자 폭증해도 ID 고갈 없음
- ✅ **FK 외래키로 명확한 관계** — JOIN 효율적
- ✅ **CARE_RECORDS.todoCount/doneCount 비정규화** — COUNT 쿼리 안 돌림
- ✅ **AUTOBIO_JOBS.sourceData JSON** — 자서전 생성 시 JOIN 0회

추가로 반드시 필요한 것 (지금은 없음)

필수	효과	누락 시
인덱스 (teacherId+date 등)	1ms vs 1초	사용자 1000명만 돼도 다운
페이지네이션 (LIMIT 강제)	30개씩 가져오기	메모리 폭발
자서전 비동기 큐	API 안 막힘	동시 10명만 자서전 누르면 멈춤

사용자 규모별 한계

사용자	현재 ERD로 가능?
100명	✅ 여유
1,000명	✅ 인덱스만 추가
10,000명	⚠️ Redis 캐싱 필요
100,000명	❌ Read Replica + 큐 필수

Q2. 새로운 탭이 추가되어도 새 구역을 가져서 병목이 안 생기는지?

> 답 — ERD 구조상 OK. 단, "새 탭 = 새 테이블" 원칙만 지키면 됨.

왜 안 생기는가

이 ERD는 **"한 도메인 = 한 테이블 묶음"** 구조라서 새 기능 추가가 격리됩니다.

```
새 탭 "건강기록" 추가 시:
  ├─ health_records 테이블 신규 생성
  ├─ 기존 테이블 0건 수정
  └─ 다른 도메인 성능 영향 0
```

새 탭 추가 체크리스트

항목	이유
1. 새 테이블만 만들기	기존 테이블 컬럼 추가 금지
2. 공통 컬럼 4개 포함	id / createdDate / modifiedDate / status
3. teacherId 또는 studentId FK	권한·소유 명확
4. 필수 인덱스 같이 생성	처음부터 빠르게
5. 페이지네이션 강제	항상 LIMIT

위험 케이스 (피해야 함)

```
❌ 기존 STUDENTS 테이블에 healthData JSON 컬럼 추가
   → 학생 명부 조회마다 무거워짐
   → 도미노로 LIFE_RECORDS, CARE_MOODS 다 느려짐

✅ 새 health_records 테이블 + studentId FK
   → 학생 명부 영향 0
```

Q3. 지금 각 백엔드 구역끼리 매핑이 잘 이루어지는지?

> 답 — 매핑은 잘 됨. 단, 일부 구간은 비정규화로 보강 필요.

매핑이 잘 된 곳 ✅

관계	평가
SCHOOLS → TEACHERS → CLASSES → STUDENTS	4단계 계층 명확
TEACHERS → CARE_RECORDS → CARE_TODOS	자서전 생성 효율적
STUDENTS ↔ CARE_MOODS (학생별 감정)	학생 자서전 가능
AUTOBIO_JOBS.sourceData	스냅샷 비정규화 잘됨

매핑이 약한 곳 ⚠️

문제	원인	해결
자서전 생성 시 N+1 쿼리 가능성	sourceData JSON에 다 안 담으면	sourceData에 모든 필요 데이터 전부 스냅샷
창체활동 ↔ 학생 미연결	CREATIVE_ACT가 classId만 FK	creative_activity_students 조인 테이블 추가
학사일정 ↔ 학급 미연결	SCHEDULES는 schoolId만 FK	학급별 일정 필요하면 classId 추가
MEALS ↔ 학생 좋아요 미연결	likeCount만 있고 누가 눌렀는지 모름	meal_likes 테이블 추가

매핑 보강 우선순위

```
🔴 P0: AUTOBIO_JOBS.sourceData에 모든 필수 데이터 통째 저장
🟡 P1: meal_likes 조인 테이블 (좋아요 누가 눌렀는지)
🟢 P2: creative_activity_students (학생별 창체 활동 추적)
```

Q4. 새 탭이 추가되면 기존 구역들과 저절로 매핑이 잘 이루어질지?

> 답 — "저절로"는 안 됨. 하지만 표준 패턴만 따르면 5분 안에 끝남.

자동 매핑이 되는 조건 (3가지 표준)

표준 1 — FK 명명 규칙

```
teacherId  → 항상 TEACHERS.teacherId 참조
studentId  → 항상 STUDENTS.studentId 참조
classId    → 항상 CLASSES.classId 참조
schoolId   → 항상 SCHOOLS.schoolId 참조
```

이 규칙만 지키면 **TypeORM/Prisma가 자동으로 관계 추론**합니다.

표준 2 — 공통 컬럼 4개 (Base Entity)

```
@Entity()
export class BaseEntity {
  @PrimaryGeneratedColumn() id: number;
  @CreateDateColumn() createdDate: Date;
  @UpdateDateColumn() modifiedDate: Date;
  @Column({ default: 'NORMAL' }) status: string;
}
```

```
// 새 탭 추가 시
@Entity()
export class HealthRecord extends BaseEntity {
  @ManyToOne(() => Student) student: Student;
  // ... 도메인 컬럼만
}
```

표준 3 — 모듈 자동 등록

```
@Module({
  imports: [
    ...기존 모듈들,
    HealthModule, // 한 줄 추가하면 끝
  ],
})
```

새 탭 추가 시 작업량 (표준 따랐을 때)

단계	시간
1. Entity 파일 생성 (FK 표준 사용)	3분
2. Module/Service/Controller 자동 생성 (nest g)	1분
3. AppModule에 imports 추가	10초
4. 마이그레이션 생성·실행	1분
합계	**5분**

자동 매핑이 깨지는 케이스

```
❌ studentId 대신 student_uuid 사용 → 관계 추론 실패
❌ Base Entity 안 쓰고 컬럼 직접 정의 → status 누락 → soft delete 안 됨
❌ Module 만들고 AppModule 등록 안 함 → API 라우팅 안 됨
```

한 줄 요약

질문	답
1. 회원 1만명?	✅ 인덱스+캐싱 추가하면 가능
2. 새 탭 격리?	✅ 한 도메인 = 한 테이블 원칙 지키면
3. 현재 매핑?	✅ 80점, sourceData 스냅샷·meal_likes만 보강
4. 새 탭 자동 매핑?	✅ FK 명명 표준 + Base Entity 지키면 5분